



L5: Linear Programming and Connections with Continuous Control

EECE 576N | From MDPs to Deep RL | Fall 2026

Cyrus Neary | cyrus.neary@ubc.ca

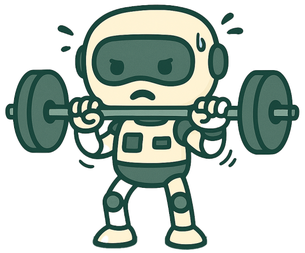
Last class

Bellman's equation and operator

$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^\pi(s') \right] \qquad (T_\pi V)(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V(s') \right]$$

Bellman's optimality equation and operator

$$V^*(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^*(s') \right] \qquad (T_* V)(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V(s') \right]$$



Policy iteration

Policy evaluation

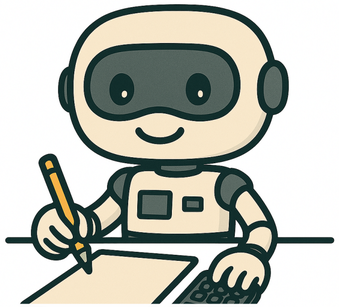
$$V_\pi = (I - \gamma P_\pi)^{-1} R_\pi$$

For k = 0, 1, 2, …

$$V_{k+1} \leftarrow T_\pi V_k$$

Policy improvement

$$\pi'(s) \in \text{argmax}_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^\pi(s') \right]$$



Value iteration

For k = 0, 1, 2, …

$$V_{k+1} \leftarrow T_* V_k$$

$$\pi^*(s) \in \text{argmax}_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^*(s') \right]$$

Linear Programming Formulations of MDPs

Linear programs

https://en.wikipedia.org/wiki/Linear_programming#Standard_form

Standard form [edit]

Standard form is the usual and most intuitive form of describing a linear programming problem. It consists of the following three parts:

- A **linear (or affine) function to be maximized**

  e.g. $f(x_1, x_2) = c_1 x_1 + c_2 x_2$

- **Problem constraints** of the following form

  e.g.

  $$\begin{aligned} a_{11} x_1 + a_{12} x_2 &\leq b_1 \\ a_{21} x_1 + a_{22} x_2 &\leq b_2 \\ a_{31} x_1 + a_{32} x_2 &\leq b_3 \end{aligned}$$

- **Non-negative variables**

  e.g.

  $$\begin{aligned} x_1 &\geq 0 \\ x_2 &\geq 0 \end{aligned}$$

The problem is usually expressed in *matrix form*, and then becomes:

$$\max\{\, \mathbf{c}^{\mathsf{T}} \mathbf{x} \mid \mathbf{x} \in \mathbb{R}^n \wedge A\mathbf{x} \leq \mathbf{b} \wedge \mathbf{x} \geq 0 \,\}$$

Other forms, such as minimization problems, problems with constraints on alternative forms, and problems involving negative variables can always be rewritten into an equivalent problem in standard form.

Decision vector x
cost vector c

$$\min_x c^T x$$
$$\text{s.t. } Ax \leq b$$
$$x \geq 0$$

Intuition and big picture

Key idea: Find the value function that satisfies Bellman optimality as a set of linear constraints.

In comparison:

- Value iteration finds a fixed point of the Bellman operator.
- LP solutions directly solve constraints with optimization algorithms.

Why study LP solutions to MDPs?

- Easy to incorporate additional constraints.
- Easy to incorporate different optimization objectives.
- Intuition on MDP solutions, and elegant theory.

Upper bound on optimal value function

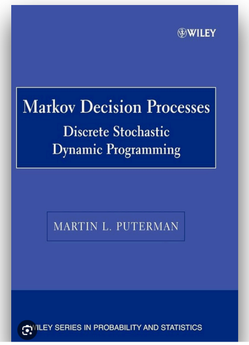
If you can find a candidate value function $f(s)$ such that

inequality constraint → Bellman equation

$$f(s) \geq R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) f(s'), \quad \forall a \in A_s, s \in S$$

Then $f(s)$ provides an upper bound on the MDP's optimal value function.

$$f(s) \geq V^*(s)$$



See Theorem 6.2.2

Primal LP Formulation

Idea: find the smallest candidate function $f(s)$, subject to the constraint that $f(s)$ upper bounds the MDP's optimal value function.

Primal LP Formulation

Idea: find the smallest candidate function $f(s)$, subject to the constraint that $f(s)$ upper bounds the MDP's optimal value function.

$$\min_{f} \sum_{s \in S} \alpha(s) f_s$$

$$\text{s.t.} \quad f_s \geq R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) f'_s, \quad \forall a \in A_s, s \in S$$

$$f_s \in \mathbb{R}^{|S|} \text{ unconstrained}$$

Primal LP Formulation

Idea: find the smallest candidate function $f(s)$, subject to the constraint that $f(s)$ upper bounds the MDP's optimal value function.

$$\min_{f} \sum_{s \in S} \alpha(s) f_s$$

$$\text{s.t.} \quad f_s \geq R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) f'_s, \quad \forall a \in A_s, s \in S$$

$$f_s \in \mathbb{R}^{|S|} \text{ unconstrained}$$

Considering MDPs with finite state and action spaces, we need to search over vectors $f \in \mathbb{R}^{|S|}$.

Solving this LP yields the MDP's optimal value function.

Dual LP formulation

It is more informative to consider the dual LP.

$$\max_{x_{s,a} \in \mathbb{R}^{|S| \cdot |A|}} \sum_{s \in S, a \in A} R(s,a) x_{s,a}$$

$$\text{s.t.} \quad \sum_{a \in A_{s'}} x_{s',a} = \alpha(s') + \gamma \sum_{s \in S, a \in A_s} T(s' \mid s, a) x_{s,a} \quad \forall a \in A_s, s \in S$$

$$x_{s,a} \geq 0, \quad \forall a \in A_s, s \in S$$

Dual linear programs

https://en.wikipedia.org/wiki/Dual_linear_program

Dual linear program

5 languages

Article Talk Read Edit View history Tools

From Wikipedia, the free encyclopedia

The **dual** of a given linear program (LP) is another LP that is derived from the original (the **primal**) LP in the following schematic way:

- Each variable in the primal LP becomes a constraint in the dual LP;
- Each constraint in the primal LP becomes a variable in the dual LP;
- The objective direction is inversed – maximum in the primal becomes minimum in the dual and vice versa.

The **weak duality theorem** states that the objective value of the dual LP at any feasible solution is always a bound on the objective of the primal LP at any feasible solution (upper or lower bound, depending on whether it is a maximization or minimization problem). In fact, this bounding property holds for the optimal values of the dual and primal LPs.

The **strong duality theorem** states that, moreover, if the primal has an optimal solution then the dual has an optimal solution too, *and the two optima are equal*.[1]

Form of the dual LP [edit]

Suppose we have the linear program:

Maximize $\mathbf{c}^T\mathbf{x}$ subject to $A\mathbf{x} \leq \mathbf{b}$, $\mathbf{x} \geq 0$.

We would like to construct an upper bound on the solution. So we create a linear combination of the constraints, with positive coefficients, such that the coefficients of $\mathbf{x}$ in the constraints are at least $\mathbf{c}^T$. This linear combination gives us an upper bound on the objective. The variables $\mathbf{y}$ of the dual LP are the coefficients of this linear combination. The dual LP tries to find such coefficients that *minimize* the resulting upper bound. This gives the following LP:[1]:81–83

Minimize $\mathbf{b}^T\mathbf{y}$ subject to $A^T\mathbf{y} \geq \mathbf{c}$, $\mathbf{y} \geq 0$

This LP is called the *dual of* the original LP.

Dual LP formulation

Interpretation: $x_{s,a}$ represents the total discounted occupancy measure under initial-state distribution $\alpha(s)$ that the system occupies state s and chooses action a.

$$\max_{x_{s,a} \in \mathbb{R}^{|S| \cdot |A|}} \sum_{s \in S, a \in A} R(s,a) x_{s,a}$$

Maximize discounted sum of total rewards.

$x \in \mathbb{R}^d$

$d = |S| \times |A|$

$$\text{s.t.} \quad \sum_{a \in A_{s'}} x_{s',a} = \alpha(s') + \gamma \sum_{s \in S, a \in A_s} T(s' \mid s, a) x_{s,a} \quad \forall a \in A_s, s \in S$$

$$x_{s,a} \geq 0, \quad \forall a \in A_s, s \in S$$

Dual LP formulation

Interpretation: $x_{s,a}$ represents the total discounted occupancy measure under initial-state distribution $\alpha(s)$ that the system occupies state s and chooses action a.

$$\max_{x_{s,a} \in \mathbb{R}^{|S| \cdot |A|}} \sum_{s \in S, a \in A} R(s,a) x_{s,a}$$

Maximize discounted sum of total rewards.

$$\text{s.t.} \sum_{a \in A_{s'}} x_{s',a} = \alpha(s') + \gamma \sum_{s \in S, a \in A_s} T(s' \mid s, a) x_{s,a} \quad \forall a \in A_s, s' \in S$$

$$x_{s,a} \geq 0, \quad \forall a \in A_s, s \in S$$

How much am I in state s'

Constraints enforce MDP dynamics: LHS is related to occupancy of s', RHS is the probability of starting in state s' summed with all probabilities of transitioning from (s,a) to s' for all $s \in S, a \in A_s$.

Dual LP formulation

Interpretation: $x_{s,a}$ represents the total discounted occupancy measure under initial-state distribution $\alpha(s)$ that the system occupies state s and chooses action a.

$$\max_{x_{s,a} \in \mathbb{R}^{|S| \cdot |A|}} \sum_{s \in S, a \in A} R(s,a) x_{s,a}$$

Maximize discounted sum of total rewards.

$$\text{s.t.} \quad \sum_{a \in A_{s'}} x_{s',a} = \alpha(s') + \gamma \sum_{s \in S, a \in A_s} T(s' \mid s, a) x_{s,a} \quad \forall a \in A_s, s \in S$$

Constraints enforce MDP dynamics: LHS is related to occupancy of s', RHS is the probability of starting in state s' summed with all probabilities of transitioning from (s,a) to s' for all $s \in S, a \in A_s$.

$$x_{s,a} \geq 0, \quad \forall a \in A_s, s \in S$$

Occupancy variables must be positive.

Connecting $x_{s,a}$ with $\pi(a \mid s)$

Interpretation: $x_{s,a}$ represents the total discounted occupancy measure under initial-state distribution $\alpha(s)$ that the system occupies state s and chooses action a.

Connecting $x_{s,a}$ with $\pi(a \mid s)$

Interpretation: $x_{s,a}$ represents the total discounted occupancy measure under initial-state distribution $\alpha(s)$ that the system occupies state s and chooses action a.

Theorem 6.9.1 (Puterman):

A. For each $\pi \in \Pi^{MR}$, $s \in S$, and $a \in A_s$ define $x^{\pi}_{s,a}$ by

$$x^{\pi}_{s,a} = \sum_{j \in S} \alpha(j) \sum_{t=1}^{\infty} \gamma^t P^{\pi}(S_t = s, A_t = a \mid S_0 = j)$$

Then $x^{\pi}_{s,a}$ is a feasible solution to the dual problem.

Connecting $x_{s,a}$ with $\pi(a \mid s)$

Interpretation: $x_{s,a}$ represents the total discounted occupancy measure under initial-state distribution $\alpha(s)$ that the system occupies state s and chooses action a.

Theorem 6.9.1 (Puterman):

A. For each $\pi \in \Pi^{MR}$, $s \in S$, and $a \in A_s$ define $x^{\pi}_{s,a}$ by

$$x^{\pi}_{s,a} = \sum_{j \in S} \alpha(j) \sum_{t=1}^{\infty} \gamma^t P^{\pi}(S_t = s, A_t = a \mid S_0 = j)$$

Then $x^{\pi}_{s,a}$ is a feasible solution to the dual problem.

B. Suppose $x_{s,a}$ is a feasible solution to the dual problem, then, for each $s \in S$, $\Sigma_{a \in A_s} x_{s,a} > 0$. Define the randomized stationary policy $\pi_x(a \mid s)$ by:

$$\pi_x(a \mid s) = \frac{x_{s,a}}{\sum_{a' \in A_s} x_{s,a'}}$$

Then $x^{\pi_x}_{s,a}$ as defined in the equation above is a feasible solution to the dual LP, and $x^{\pi_x}_{s,a} = x_{s,a}$ for all $a \in A_s$ and $s \in S$.

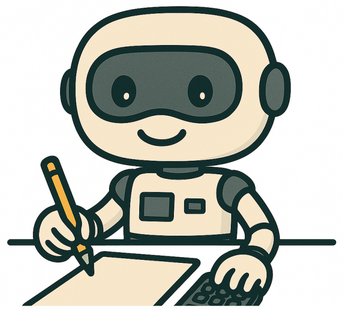
In words, what does this theorem tell us?

- Direct correspondance between $x_{s,a}$ and Markovian randomized policies.
- From any feasible soln to the dual LP, we can extract a policy.
- Given a policy π, there is a corresponding vector x that is feasible to the dual LP.

Under the given interpretation of $x_{s,a}$, what does the dual objective $\sum_{s \in S} \sum_{a \in A_s} R(s,a) x_{s,a}$ represent?

$$\sum_s \sum_a R(s,a) x_{s,a} = \sum_s \sum_a \sum_{j \in S} \alpha(j) \sum_{t=1}^{\infty} \gamma^t P(S_t = s, A_t = a \mid S_0 = j) R(s,a)$$

$$= \sum_s \alpha(s) V^{\pi_x}(s)$$



How can we use the results of this theorem to find policies?

First, solve the dual LP

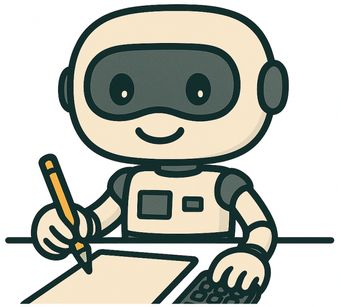
Then $\pi^*(a|s) = \dfrac{x^*_{s,a}}{\sum_{a'} x^*_{s,a'}}$

What kinds of additional constraints can we model with the dual problem?

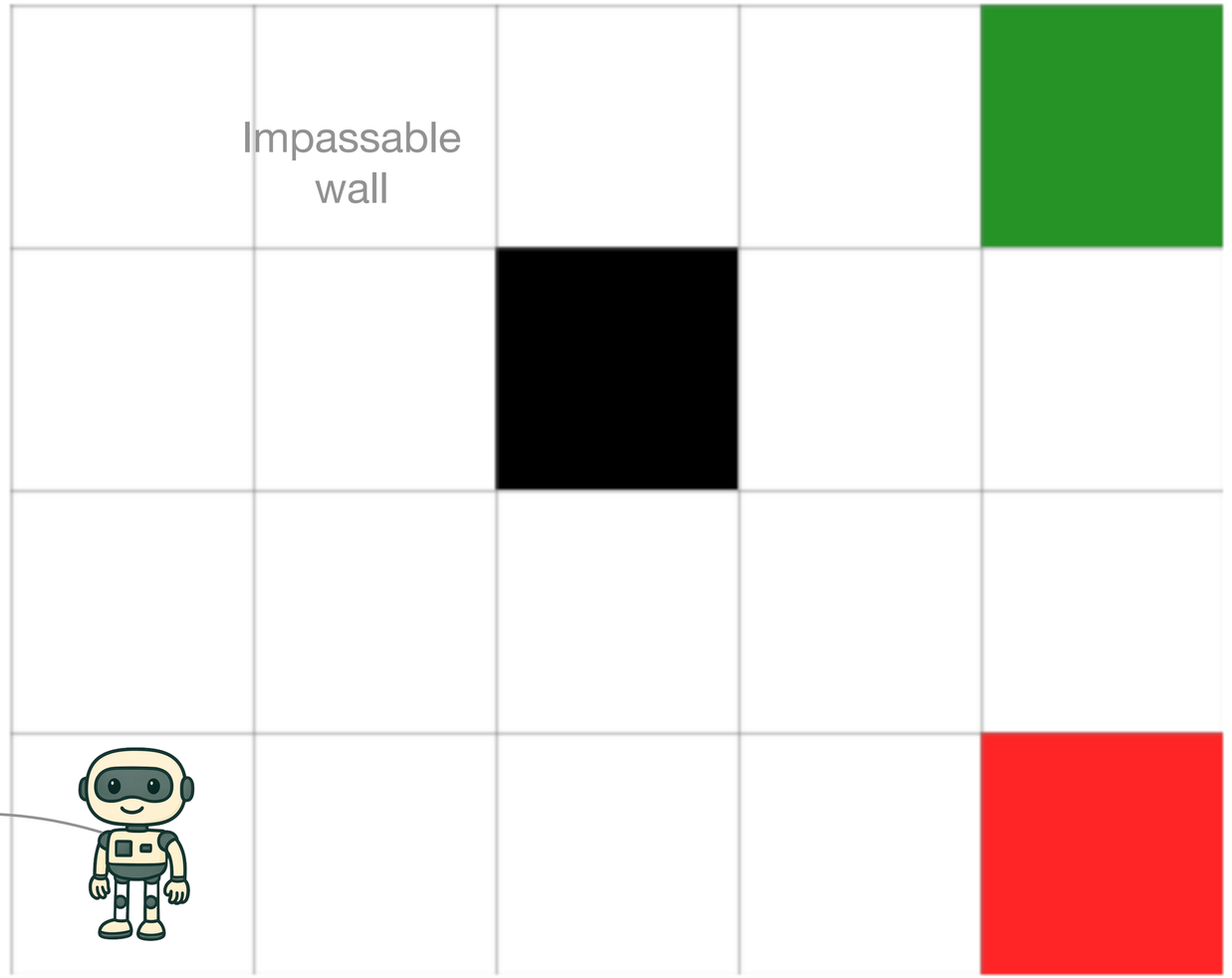
- Constraints on discounted occupancy for particular state-action pairs.

What if we wanted to solve undiscounted finite horizon problems? Can we still use the dual LP?

- Yes, add time as an extra dimension to each state variable $x_{s,a,t}$ $\forall s, a, t \in \{0, 1, \cdots, T\}$



Worked example



LP Solution

$$\max_{x_{s,a} \in \mathbb{R}^{|S| \cdot |A|}} \sum_{s \in S, a \in A} R(s,a) x_{s,a}$$

$$\sum_{a \in A_{s'}} x_{s',a} = \alpha(s') + \gamma \sum_{s \in S, a \in A_s} T(s' \mid s, a) x_{s,a} \quad \forall a \in A_s, s \in S$$

$$x_{s,a} \geq 0, \quad \forall a \in A_s, s \in S$$

Solve using scipy.optimize.linprog()

LP Solution

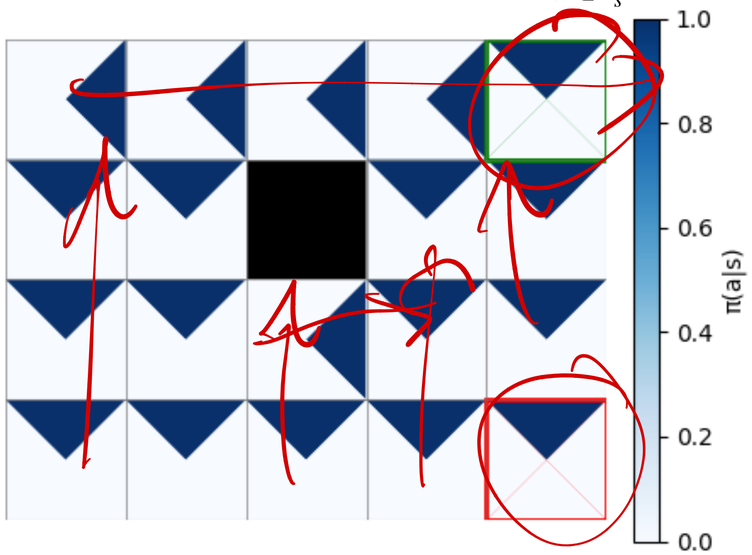
$$\max_{x_{s,a} \in \mathbb{R}^{|S| \cdot |A|}} \sum_{s \in S, a \in A} R(s,a) x_{s,a}$$

$$\sum_{a \in A_{s'}} x_{s',a} = \alpha(s') + \gamma \sum_{s \in S, a \in A_s} T(s' \mid s, a) x_{s,a} \quad \forall a \in A_s, s \in S$$

$$x_{s,a} \geq 0, \quad \forall a \in A_s, s \in S$$

Solve using scipy.optimize.linprog()

Optimal policy $\pi_x(a \mid s) = \dfrac{x_{s,a}}{\sum_{a' \in A_s} x_{s,a'}}$



LP Solution

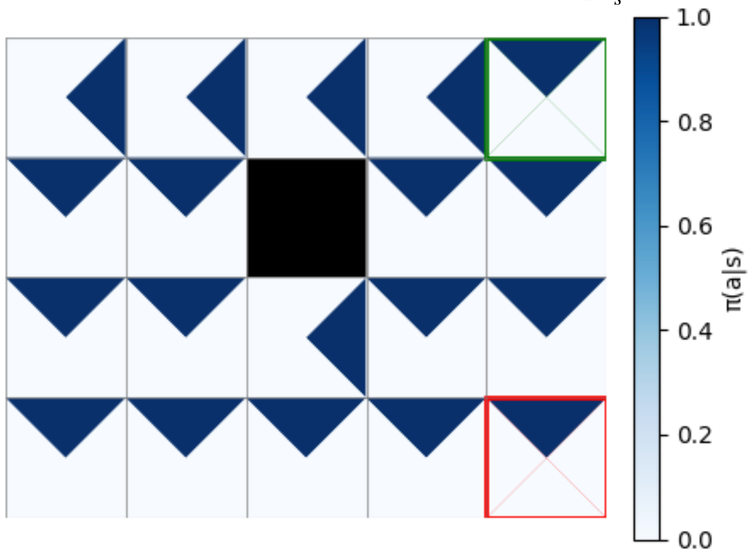
$$\max_{x_{s,a} \in \mathbb{R}^{|S| \cdot |A|}} \sum_{s \in S, a \in A} R(s,a) x_{s,a}$$

$$\sum_{a \in A_{s'}} x_{s',a} = \alpha(s') + \gamma \sum_{s \in S, a \in A_s} T(s' \mid s, a) x_{s,a} \quad \forall a \in A_s, s \in S$$

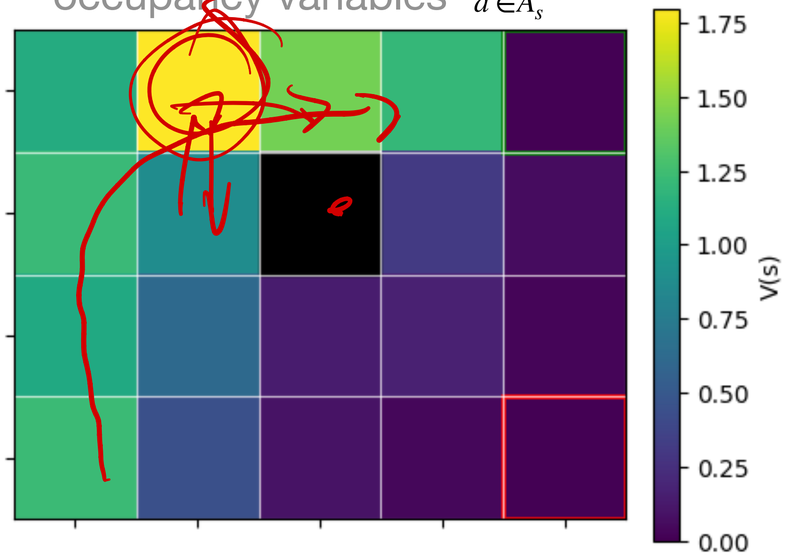
$$x_{s,a} \geq 0, \quad \forall a \in A_s, s \in S$$

Solve using scipy.optimize.linprog()

Optimal policy $\pi_x(a \mid s) = \dfrac{x_{s,a}}{\sum_{a' \in A_s} x_{s,a'}}$



Discounted state-occupancy variables $\sum_{a' \in A_s} x_{s,a'}$



LP Solution with constraints

$$\max_{x_{s,a} \in \mathbb{R}^{|S| \cdot |A|}} \sum_{s \in S, a \in A} R(s,a) x_{s,a}$$

$$\sum_{a \in A_{s'}} x_{s',a} = \alpha(s') + \gamma \sum_{s \in S, a \in A_s} T(s' \mid s, a) x_{s,a} \quad \forall a \in A_s, s \in S$$

$$x_{s,a} \geq 0, \quad \forall a \in A_s, s \in S$$

$$x_{s,a} \leq 0.1, \quad \forall a \in A_s, s \in S_{constraint}$$

Add constraint enforcing low occupancy in a state a long the agent's previously planned path.

LP Solution with constraints

$$\max_{x_{s,a} \in \mathbb{R}^{|S| \cdot |A|}} \sum_{s \in S, a \in A} R(s,a) x_{s,a}$$

$$\sum_{a \in A_{s'}} x_{s',a} = \alpha(s') + \gamma \sum_{s \in S, a \in A_s} T(s' \mid s, a) x_{s,a} \quad \forall a \in A_s, s \in S$$

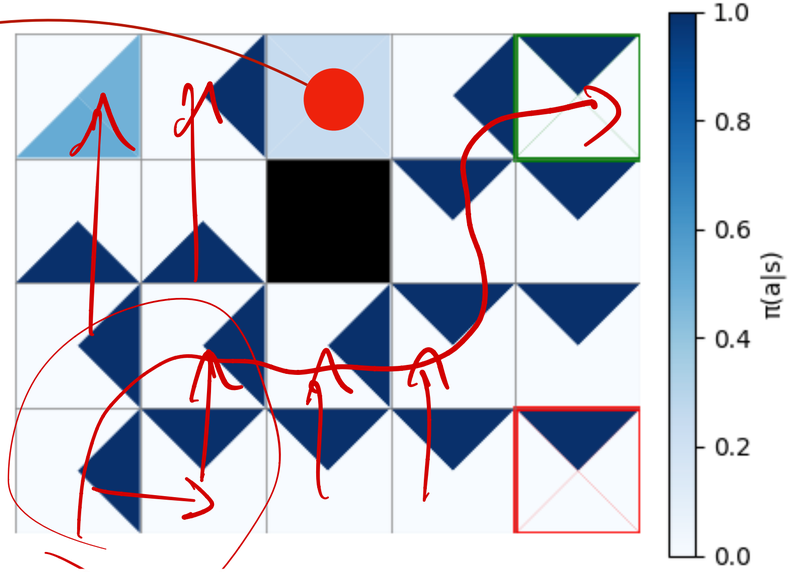
$$x_{s,a} \geq 0, \quad \forall a \in A_s, s \in S$$

$$x_{s,a} \leq 0.1, \quad \forall a \in A_s, s \in S_{constraint}$$

Add constraint enforcing low occupancy in a state a long the agent's previously planned path.

Optimal policy $\pi_x(a \mid s) = \dfrac{x_{s,a}}{\sum_{a' \in A_s} x_{s,a'}}$

$x_{s,a} \leq 0.1$



LP Solution with constraints

$$\max_{x_{s,a} \in \mathbb{R}^{|S| \cdot |A|}} \sum_{s \in S, a \in A} R(s,a) x_{s,a}$$

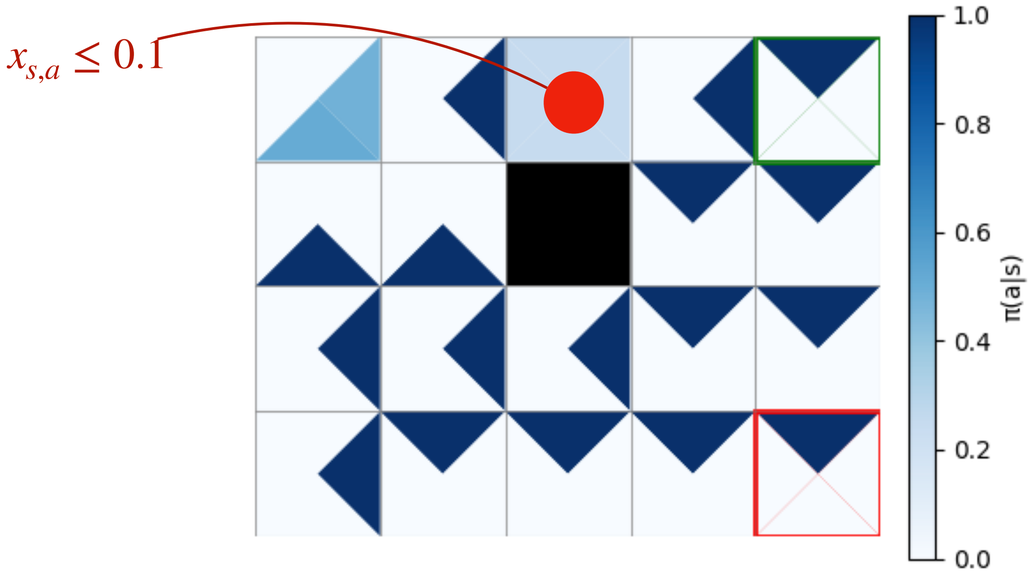
$$\sum_{a \in A_{s'}} x_{s',a} = \alpha(s') + \gamma \sum_{s \in S, a \in A_s} T(s' \mid s, a) x_{s,a} \quad \forall a \in A_s, s \in S$$

$$x_{s,a} \geq 0, \quad \forall a \in A_s, s \in S$$

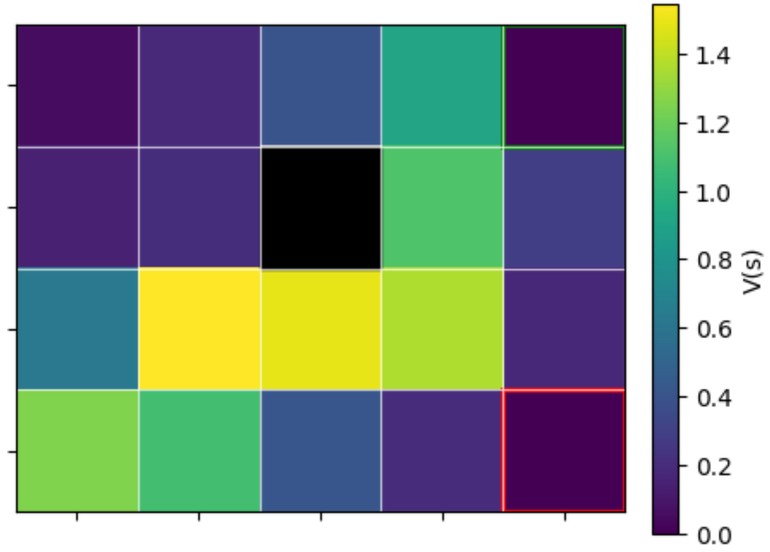
$$x_{s,a} \leq 0.1, \quad \forall a \in A_s, s \in S_{constraint}$$

Add constraint enforcing low occupancy in a state a long the agent's previously planned path.

Optimal policy $\pi_x(a \mid s) = \dfrac{x_{s,a}}{\sum_{a' \in A_s} x_{s,a'}}$



Discounted state-occupancy variables $\sum_{a' \in A_s} x_{s,a'}$



The dual LP offers flexibility to formulate/solve NEW types of problems.

Planning Not to Talk: Multiagent Systems that are Robust to Communication Loss

Mustafa O. Karabag*
The University of Texas at Austin
Texas, USA
karabag@utexas.edu

Cyrus Neary*
The University of Texas at Austin
Texas, USA
cneary@utexas.edu

Ufuk Topcu
The University of Texas at Austin
Texas, USA
utopcu@utexas.edu

ABSTRACT

In a cooperative multiagent system, a collection of agents executes a joint policy in order to achieve some common objective. The successful deployment of such systems hinges on the availability of reliable inter-agent communication. However, many sources of potential disruption to communication exist in practice, such as radio interference, hardware failure, and adversarial attacks. In this work, we develop joint policies for cooperative multiagent systems that are robust to potential losses in communication. More specifically, we develop joint policies for cooperative Markov games with reach-avoid objectives. First, we propose an algorithm for the decentralized execution of joint policies during periods of communication loss. Next, we use the total correlation of the state-action process induced by a joint policy as a measure of the intrinsic dependencies between the agents. We then use this measure to lower-bound the performance of a joint policy when communication is lost. Finally, we present an algorithm that maximizes a proxy to this lower bound in order to synthesize minimum-dependency joint policies that are robust to communication loss. Numerical experiments show that the proposed minimum-dependency policies require minimal coordination between the agents while incurring little to no loss in performance; the total correlation value of the synthesized policy is one fifth of the total correlation value of the baseline policy

systems — such as the coordination of autonomous vehicles, the control of networks of mobile sensors, or the co — can be modeled as collections of interacting

Inter-agent communication plays an esser cessful deployment of such multiagent systen coordination between agents via communicatio upon the particular actions to collectively take in time – is often necessary for the successful in optimal joint policy [7]. However, many poss munication disruption exist in practice, such a hardware failure, or even adversarial attacks i the team. Lost or unreliable communication ca tial degradation of the team's performance, bec agents' ability to coordinate. Despite this rel performance on communication, multiagent lea algorithms typically do not offer robustness possible losses in communication.

In this work, we study multiagent systems th communication losses. In detail, we study se decision problems formulated as cooperative reach-avoid objectives [3, 19]. We make the foll

(1) Imaginary Play for Policy Execution Duri munication. During periods of lost communicati tains imaginary versions of their teammates

Given that the stationary policies suffice, (1) in the main body of the paper can be rewritten in terms of the occupancy measures:

$$\max_{x} \quad v^{full} - \delta l^{full} - \beta \left(\sum_{i=1}^{N} H(\bar{X}^i) - H(X) \right) \tag{25a}$$

$$\text{s.t.} \quad v^{full} = \sum_{\substack{s \in \mathcal{S} \setminus (\mathcal{S}_{\mathcal{T}} \cup \mathcal{S}_{\mathcal{D}}) \\ a \in \mathcal{A} \\ y \in \mathcal{S}_{\mathcal{T}}}} x_{s,a} \mathcal{T}(s,a,y) \tag{25b}$$

$$l^{full} = \sum_{\substack{s \in \mathcal{S} \\ a \in \mathcal{A} \cup \{\epsilon\}}} x_{s,a} \tag{25c}$$

$$H(X) = \sum_{\substack{s \in \mathcal{S} \\ a \in \mathcal{A}}} x_{s,a} \log \left(\frac{\sum_{b \in \mathcal{A}} x_{s,b}}{x_{s,a}} \right) + \sum_{\substack{s \in \mathcal{S} \\ a \in \mathcal{A}}} x_{s,a} \sum_{y \in \mathcal{S}} \mathcal{T}(s,a,y) \log \left(\frac{1}{\mathcal{T}(s,a,y)} \right) \tag{25d}$$

$$H(\bar{X}^i) = \sum_{\substack{s^i \in \mathcal{S}^i \\ a^i \in \mathcal{A}^i \cup \{\epsilon^i\}}} x_{s^i,a^i} \log \left(\frac{\sum_{b^i \in \mathcal{A}^i} x_{s^i,b^i}}{x_{s^i,a^i}} \right) + \sum_{\substack{s^i \in \mathcal{S}^i \\ a^i \in \mathcal{A}^i \cup \{\epsilon^i\}}} x_{s^i,a^i} \sum_{y^i \in \mathcal{S}^i \cup \{s^i_\epsilon\}} \mathcal{T}^i(s^i,a^i,y^i) \log \left(\frac{1}{\mathcal{T}^i(s^i,a^i,y^i)} \right). \tag{25e}$$

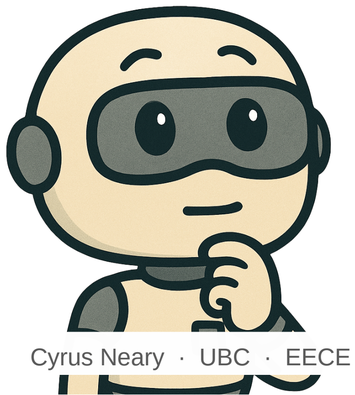
$$\sum_{a \in \mathcal{A} \cup \{\epsilon\}} x_{s,a} = \sum_{\substack{y \in \mathcal{S} \\ b \in \mathcal{A} \cup \{\epsilon\}}} x_{y,b} \mathcal{T}(y,b,s) + \mathbb{1}_{\{s_I = s\}}, \ \forall s \in \mathcal{S} \tag{25f}$$

$$x_{s,a} \geq 0, \ \forall s \in \mathcal{S}, a \in \mathcal{A} \cup \{\epsilon\} \tag{25g}$$

$$x_{s_\epsilon,a} = 0, \ \forall a \in \mathcal{A}. \tag{25h}$$

The dual variable $x_{s,a}$ represents…

A: The probability the agent is in state s and takes action a, at any timestep.
B: The expected discounted number of times the agent visits s and takes a, over the whole trajectory.
C: The expected reward collected from state s under action a.
D: The probability that a trajectory starting at s ever takes action a.
E: I don’t understand.



Given that $\sum_s \alpha(s) = 1$, what is $\sum_{s,a} x_{s,a}$ at any feasible point of the dual LP?

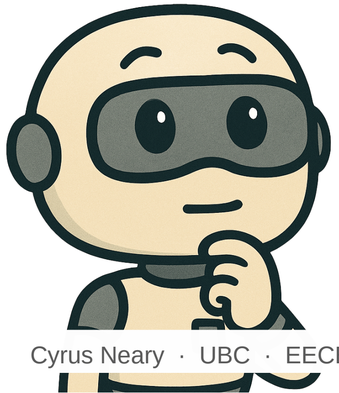
A: $\frac{1}{1-\gamma}$

B: 1

C: $|S| \cdot |A|$

D: It depends on the policy.

E: I don't understand.



what is $\sum_{s,a} x_{s,a}$ for feasible point of dual LP?

$$\sum_{a'} x_{s',a'} = \alpha(s') + \gamma \sum_{s,a} T(s'|s,a) x_{s,a}$$

$$\underbrace{\sum_{s'} \sum_{a'} x_{s',a'}}_{X} = \underbrace{\sum_{s'} \alpha(s)}_{=1} + \gamma \underbrace{\sum_{s',s,a} T(s'|s,a) x_{s,a}}_{\sum_{s,a} x_{s,a} = X}$$

$$X = 1 + \gamma X$$

$$X(1-\gamma) = 1$$

$$X = \frac{1}{1-\gamma}$$

Which of the following could NOT be added to the dual LP as a linear constraint?

A: Spend at most 0.5 in discounted occupancy measure in a designated set of hazard states S_{hazard}. i.e. $\sum_a X_{s,a} \leq 0.5$ for every $s \in S_{hazard}$.

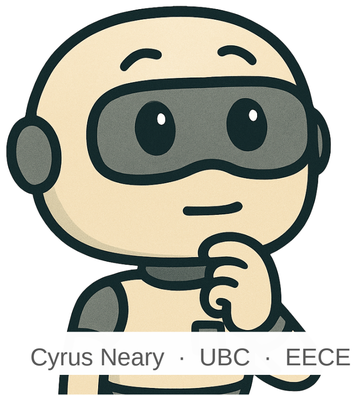
B: Visit a designated "checkpoint" state $\bar{s}$ with at least some discounted occupancy measure ($\sum_a x_{\bar{s},a} \geq 0.2$).

C: The resulting policy must be deterministic.

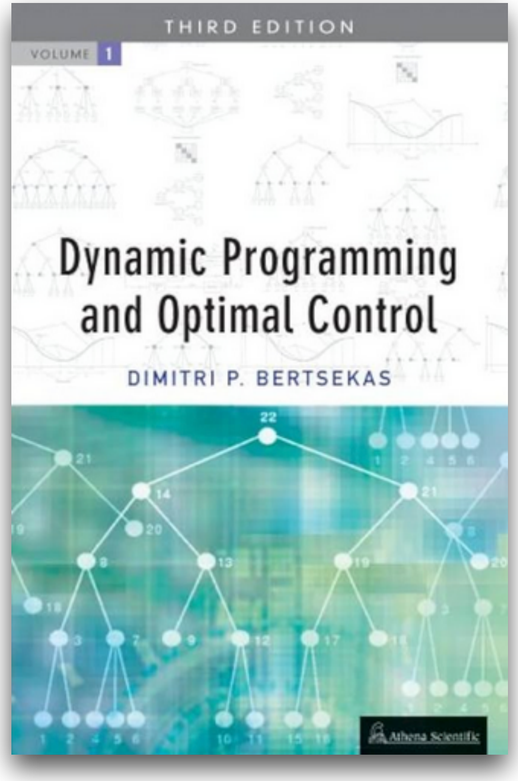
D: Keep the total expected discounted cost (where cost is a function $C(s, a)$) under a pre-specified budget.

E: I don't understand.

$$\pi(a|s) = \frac{x_{s,a}}{\sum_{a'} x_{s,a'}}$$

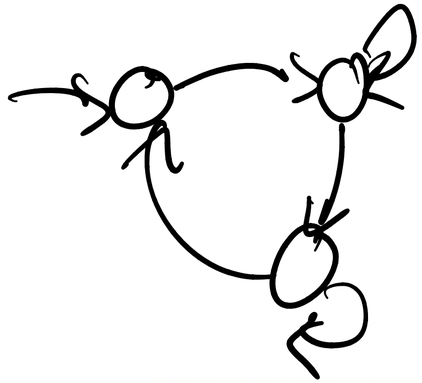


Dynamic programming in continuous control: Hamilton-Jacobi-Bellman Equations & LQR

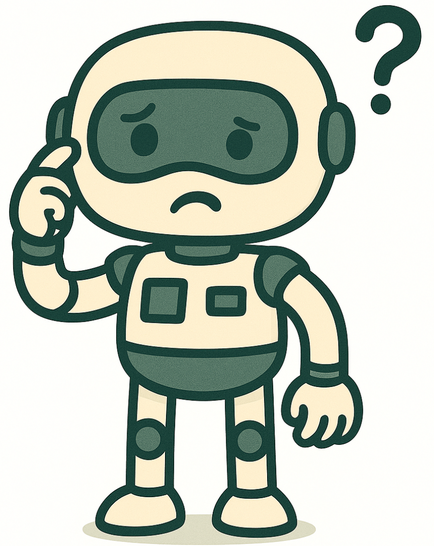


Today: Chapter 3

So far we've seen Bellman's equations for discrete-time systems with finite states and actions.



How do similar dynamic programming ideas show up in continuous-time optimal control problems with compact state and action sets?



Problem setting

Consider a continuous-time dynamic system

State vector $\in \mathbb{R}^n$

Control vector $\in \mathbb{R}^m$

Terminal time

$$\dot{x}(t) = f(x(t), u(t)), \quad 0 \leq t \leq T$$

$$x(0) = x_0$$

Initial condition

Problem setting

Consider a continuous-time dynamic system

State vector $\in \mathbb{R}^n$

Control vector $\in \mathbb{R}^m$

Terminal time

$$\dot{x}(t) = f(x(t), u(t)), \quad 0 \leq t \leq T$$

$$x(0) = x_0$$

Initial condition

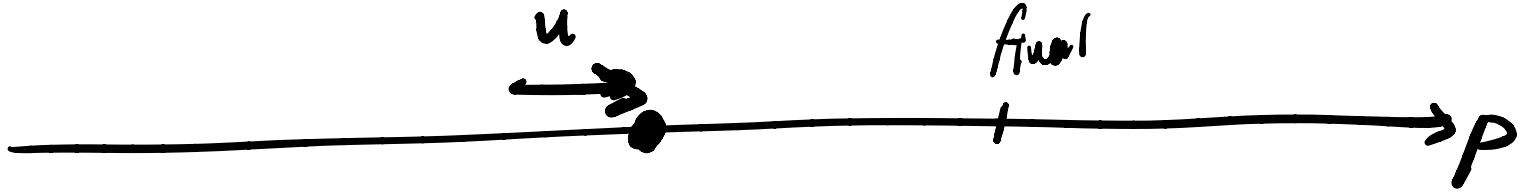
Objective is to minimize a cost function of the form

$$\min_{u(t)} \; h(x(T)) + \int_0^T g(x(t), u(t))dt$$

Terminal cost

Running cost

Example A mass with $m=1$, on a line, under the influence of a force u.



Let $x_1(t)$ be the position at time t.
Let $x_2(t)$ be the velocity.

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}$$

We can define some initial condition $x_0 = \begin{bmatrix} x_1(0) \\ x_2(0) \end{bmatrix}$

Objective Bring the mass to final position with zero vel

$$x_F = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Dynamics: $\dot{x}(t) = f(x, u)$ ← $f(x,u)$

$$\dot{x}(t) = \begin{bmatrix} \dot{x}_1(t) \\ \dot{x}_2(t) \end{bmatrix} = \begin{bmatrix} x_2(t) \\ u(t) \end{bmatrix}$$

Cost: $h(x(T)) = |x_1(T) - x_{F,1}|^2 + |x_2(T) - x_{2,F}|^2$

The Hamilton-Jacobi-Bellman PDE

Optimal “cost-to-go” function

$$\nabla_t V^*(t,x) = -\min_{u \in U} \left[g(x,u) + \nabla_x V^*(t,x) \cdot f(x,u) \right]$$

The Hamilton-Jacobi-Bellman PDE

Optimal “cost-to-go” function

$$\nabla_t V^*(t,x) = -\min_{u \in U} \left[g(x,u) + \nabla_x V^*(t,x) \cdot f(x,u) \right]$$

Change in V^* at state x
over infinitesimal dt

The Hamilton-Jacobi-Bellman PDE

Optimal “cost-to-go” function

$$\nabla_t V^*(t,x) = -\min_{u \in U} \left[g(x,u) + \nabla_x V^*(t,x) \cdot f(x,u) \right]$$

Change in V^* at state x over infinitesimal dt

Minimization problem over infinitesimal dt

The Hamilton-Jacobi-Bellman PDE

Optimal “cost-to-go” function

$$\nabla_t V^*(t,x) = -\min_{u \in U} \left[g(x,u) + \nabla_x V^*(t,x) \cdot f(x,u) \right]$$

Change in V^* at state x over infinitesimal dt

Minimization problem over infinitesimal dt

Optimal value at the resulting infinitesimally changed state x

How does this relate to the Bellman equations from last class?
Why does the continuous setting yield a differential equation?

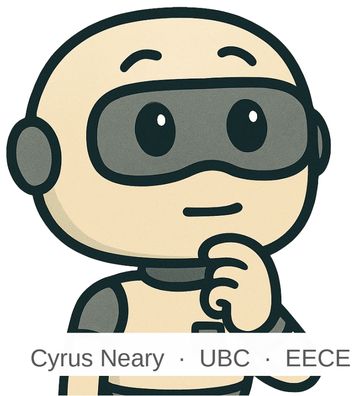
$$\nabla_t V^*(t,x) = -\min_{u \in U} \left[g(x,u) + \nabla_x V^*(t,x) \cdot f(x,u) \right]$$

vs.

$$V^*(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^*(s') \right]$$

- In discrete settings, Bellman equations tell us recurrent relations over states with discrete jumps in state (which we model with discrete Δt)

- HJB equations tell us the same thing as $\Delta t \to 0$



Deriving the Hamilton-Jacobi-Bellman PDE

Step 1: Discretize the system in time.

Original system $\dot{x}(t) = f(x(t), u(t)), \quad 0 \leq t \leq T \qquad x(0) = x_0$

Cost function $h(x(T)) + \int_0^T g(x(t), u(t))dt$

Deriving the Hamilton-Jacobi-Bellman PDE

Step 1: Discretize the system in time.

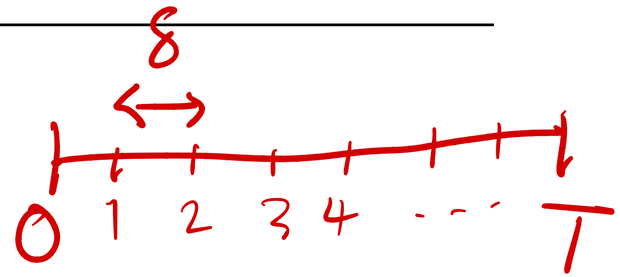
Original system $\dot{x}(t) = f(x(t), u(t)), \quad 0 \le t \le T \qquad x(0) = x_0$

Cost function $h(x(T)) + \int_0^T g(x(t), u(t))dt$

Define $\delta = \frac{T}{N}$

$x_k = x(k\delta), \quad k = 0,1,2,\ldots,N$

$u_k = u(k\delta), \quad k = 0,1,2,\ldots,N$



Deriving the Hamilton-Jacobi-Bellman PDE

Step 1: Discretize the system in time.

Original system $\dot{x}(t) = f(x(t), u(t)), \quad 0 \le t \le T \qquad x(0) = x_0$

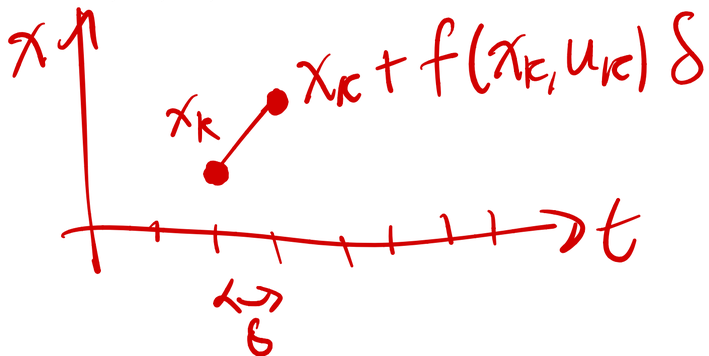
Cost function $h(x(T)) + \int_0^T g(x(t), u(t))dt$

Define $\delta = \frac{T}{N}$ $\qquad x_k = x(k\delta), \quad k = 0,1,2,\ldots,N$

$u_k = u(k\delta), \quad k = 0,1,2,\ldots,N$

Discrete system $x_{k+1} = x_k + f(x_k, u_k) \cdot \delta, \quad k = 0,1,2,\ldots N-1$

Discrete cost $h(x_N) + \sum_{k=1}^{N-1} g(x_k, u_k) \cdot \delta$



Deriving the Hamilton-Jacobi-Bellman PDE

Step 2: Apply Bellman optimality equations to discrete system.

$$\tilde{V}^*(N\delta, x) = h(x)$$ ——— Terminal cost

Deriving the Hamilton-Jacobi-Bellman PDE

Step 2: Apply Bellman optimality equations to discrete system.

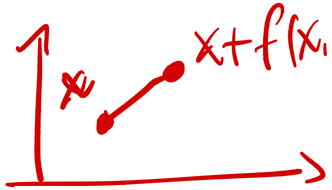
$$\tilde{V}^*(N\delta, x) = h(x) \quad \text{Terminal cost}$$

$$\tilde{V}^*(k\delta, x) = \min_{u \in U} \left[g(x,u) \cdot \delta + \tilde{V}^*(k\delta + \delta, x + f(x,u) \cdot \delta) \right], \quad k = 0, \ldots, N-1$$

Discrete optimality equations

Discrete cost-to-go function

time $= k\delta + \delta$



Note! No expectations because no stochasticity in equations.

Deriving the Hamilton-Jacobi-Bellman PDE

Step 3: Taylor expand $\tilde{V}^*$ about the point $(k\delta, x)$ in time,

(assuming required differentiability properties on $\tilde{V}^*$)

$$\tilde{V}^*(k\delta + \delta, x + f(x,u)\cdot\delta) = \tilde{V}^*(k\delta, x) + \nabla_t \tilde{V}^*(k\delta, x)\cdot\delta + \nabla_x \tilde{V}^*(k\delta, x)\cdot f(x,u)\cdot\delta + o(\delta)$$

Where $o(\delta)$ represents second-order terms satisfying $lim_{\delta\to 0}\frac{o(\delta)}{\delta} = 0$

Deriving the Hamilton-Jacobi-Bellman PDE

Step 4: Plug the Taylor expansion into discrete Bellman equation from before.

$$\tilde{V}^*(k\delta, x) = \min_{u \in U} \left[g(x,u) \cdot \delta + \tilde{V}^*(k\delta + \delta, x + f(x,u) \cdot \delta) \right], \quad k = 0, \ldots, N-1$$

Deriving the Hamilton-Jacobi-Bellman PDE

Step 4: Plug the Taylor expansion into discrete Bellman equation from before.

$$\tilde{V}^*(k\delta, x) = \min_{u \in U} \left[g(x,u) \cdot \delta + \tilde{V}^*(k\delta + \delta, x + f(x,u) \cdot \delta) \right], \quad k = 0, \ldots, N-1$$

$$\tilde{V}^*(k\delta, x) = \min_{u \in U} [g(x,u) \cdot \delta + \tilde{V}^*(k\delta, x) + \nabla_t \tilde{V}^*(k\delta, x) \cdot \delta$$
$$\nabla_x \tilde{V}^*(k\delta, x) \cdot f(x,u) \cdot \delta + o(\delta)], \quad k = 0, \ldots, N-1$$

Deriving the Hamilton-Jacobi-Bellman PDE

Step 5: Subtract $\tilde{V}^*(k\delta, x)$ from both sides, and take the limit as $\delta \to 0$.

$$\tilde{V}^*(k\delta, x) = \min_{u \in U}[g(x,u) \cdot \delta + \tilde{V}^*(k\delta, x) + \nabla_t \tilde{V}^*(k\delta, x) \cdot \delta$$
$$\nabla_x \tilde{V}^*(k\delta, x) \cdot f(x,u) \cdot \delta + o(\delta)], \quad k = 0, \ldots, N-1$$

Deriving the Hamilton-Jacobi-Bellman PDE

Step 5: Subtract $\tilde{V}^*(k\delta, x)$ from both sides, and take the limit as $\delta \to 0$.

$$\tilde{V}^*(k\delta, x) = \min_{u \in U}[g(x,u) \cdot \delta + \tilde{V}^*(k\delta, x) + \nabla_t \tilde{V}^*(k\delta, x) \cdot \delta$$

$$\nabla_x \tilde{V}^*(k\delta, x) \cdot f(x,u) \cdot \delta + o(\delta)], \quad k = 0, \ldots, N-1$$

Deriving the Hamilton-Jacobi-Bellman PDE

Step 5: Subtract $\tilde{V}^*(k\delta, x)$ from both sides, and take the limit as $\delta \to 0$.

$$\tilde{V}^*(k\delta, x) = \min_{u \in U}[g(x,u) \cdot \delta + \tilde{V}^*(k\delta, x) + \nabla_t \tilde{V}^*(k\delta, x) \cdot \delta$$

$$\nabla_x \tilde{V}^*(k\delta, x) \cdot f(x,u) \cdot \delta + o(\delta)], \quad k = 0, \ldots, N-1$$

Assume $\lim_{\delta \to 0, k \to \infty, k\delta \to t} \tilde{V}^*(k\delta, x) = V^*(t, x), \quad$ for all t, x

Deriving the Hamilton-Jacobi-Bellman PDE

Step 5: Subtract $\tilde{V}^*(k\delta, x)$ from both sides, and take the limit as $\delta \to 0$.

$$\tilde{V}^*(k\delta, x) = \min_{u \in U} [g(x,u) \cdot \delta + \tilde{V}^*(k\delta, x) + \nabla_t \tilde{V}^*(k\delta, x) \cdot \delta$$
$$\nabla_x \tilde{V}^*(k\delta, x) \cdot f(x,u) \cdot \delta + o(\delta)], \quad k = 0, \ldots, N-1$$

Assume $\lim_{\delta \to 0, k \to \infty, k\delta \to t} \tilde{V}^*(k\delta, x) = V^*(t, x), \quad$ for all t, x

$$0 = \min_{u \in U} \left[g(x,u) \cdot \delta + \nabla_t \tilde{V}^*(k\delta, x) \cdot \delta + \nabla_x \tilde{V}^*(k\delta, x) \cdot f(x,u) \cdot \delta + o(\delta) \right], \quad k = 0, \ldots, N-1$$

Deriving the Hamilton-Jacobi-Bellman PDE

Step 5: Subtract $\tilde{V}^*(k\delta, x)$ from both sides, and take the limit as $\delta \to 0$.

$$\tilde{V}^*(k\delta, x) = \min_{u \in U}[g(x,u) \cdot \delta + \tilde{V}^*(k\delta, x) + \nabla_t \tilde{V}^*(k\delta, x) \cdot \delta$$

$$\nabla_x \tilde{V}^*(k\delta, x) \cdot f(x,u) \cdot \delta + o(\delta)], \quad k = 0, \ldots, N-1$$

Assume $\lim_{\delta \to 0, k \to \infty, k\delta \to t} \tilde{V}^*(k\delta, x) = V^*(t,x), \quad$ for all t, x

$$0 = \min_{u \in U} \left[g(x,u) \cdot \delta + \nabla_t \tilde{V}^*(k\delta, x) \cdot \delta + \nabla_x \tilde{V}^*(k\delta, x) \cdot f(x,u) \cdot \delta + o(\delta) \right], \quad k = 0, \ldots, N-1$$

$\tilde{V}^*(t,x)$ $\tilde{V}^*(t,x)$ 0

Divide by δ and re-arrange

$$\Longrightarrow \boxed{\nabla_t V^*(t,x) = -\min_{u \in U} \left[g(x,u) + \nabla_x V^*(t,x) \cdot f(x,u) \right]}$$

How can we use the HJB PDE to solve control problems?

$$\nabla_t V^*(t,x) = -\min_{u \in U} \left[g(x,u) + \nabla_x V^*(t,x) \cdot f(x,u) \right]$$

Proposition 3.2.1: (Sufficiency Theorem) Suppose $V(t,x)$ is a solution to the HJB equation; that is, V is continuously differentiable in t and x, and is such that

$$0 = \min_{u \in U} \left[g(x,u) + \nabla_t V(t,x) + \nabla_x V(t,x)' f(x,u) \right], \quad \text{for all } t, x, \tag{3.2}$$

$$V(T,x) = h(x), \quad \text{for all } x. \tag{3.3}$$

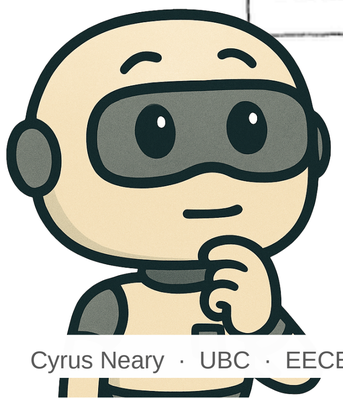
Suppose also that $\mu^*(t,x)$ attains the minimum in Eq. (3.2) for all t and x. Let $\{x^*(t) \mid t \in [0,T]\}$ be the state trajectory obtained from the given initial condition $x(0)$ when the control trajectory $u^*(t) = \mu^*\big(t, x^*(t)\big)$, $t \in [0,T]$ is used [that is, $x^*(0) = x(0)$ and for all $t \in [0,T]$, $\dot{x}^*(t) = f\big(x^*(t), \mu^*(t, x^*(t))\big)$; we assume that this differential equation has a unique solution starting at any pair (t,x) and that the control trajectory $\{\mu^*(t, x^*(t)) \mid t \in [0,T]\}$ is piecewise continuous as a function of t]. Then V is equal to the optimal cost-to-go function, i.e.,

$$V(t,x) = J^*(t,x), \quad \text{for all } t, x.$$

Furthermore, the control trajectory $\{u^*(t) \mid t \in [0,T]\}$ is optimal.

In discrete settings: Value iteration to solve for V^*, and then solve one-step minization problems using V^* to get optimal policy.

Now: Solve HJB PDE to get $V(t,x)$. P/u into (3.2) and solve minimization problem to get optimal control policy $u^*(t)$.



Example: Linear-Quadratic Problems

Consider the n-dimensional linear system

$$\dot{x}(t) = Ax(t) + Bx(t)$$

$$A \in \mathbb{R}^{n \times n}, B \in \mathbb{R}^{n \times m}$$

and the quadratic cost function

$$x(T)'Q_T x(T) + \int_0^T \left(x(t)'Qx(t) + u(t)'Ru(t) \right) dt$$

$$Q_T, Q \in \mathbb{R}^{n \times n}$$ Symmetric positive semidefinite

$$R \in \mathbb{R}^{m \times m}$$ Symmetric positive definite

The HJB equations become

$$V(T, x) = x'Q_T x$$

$$0 = \min_{u \in \mathbb{R}^{m \times m}} \left[x'Qx + u'Ru + \nabla_t V(t, x) + \nabla_x V(t, x)'(Ax + Bu) \right]$$

Example: Linear-Quadratic Problems

Try a solution of the form

$$V(t,x) = x'K(t)x, \quad \text{for some symmetric } K(t) \in \mathbb{R}^{n\times n}$$

Example: Linear-Quadratic Problems

Try a solution of the form

$$V(t,x) = x'K(t)x, \quad \text{for some symmetric } K(t) \in \mathbb{R}^{n\times n}$$

Then we have $\nabla_x V(t,x) = 2K(t)x$ and $\nabla_t V(t,x) = x'\dot{K}(t)x$

Example: Linear-Quadratic Problems

Try a solution of the form

$$V(t,x) = x'K(t)x, \quad \text{for some symmetric } K(t) \in \mathbb{R}^{n\times n}$$

Then we have $\nabla_x V(t,x) = 2K(t)x$ and $\nabla_t V(t,x) = x'\dot{K}(t)x$

Substituting into our HJB equations we get

$$V(T,x) = x'Q_T x$$

$$0 = \min_{u\in\mathbb{R}^{m\times m}} \left[x'Qx + u'Ru + x'\dot{K}(t)x + 2x'K(t)Ax + 2x'K(t)Bu\right]$$

Example: Linear-Quadratic Problems

$$V(T, x) = x' Q_T x$$

$$0 = \min_{u \in \mathbb{R}^{m \times m}} \left[x' Q x + u' R u + x' \dot{K}(t) x + 2 x' K(t) A x + 2 x' K(t) B u \right]$$

Solve the minimization in u by setting the derivative to zero.

Example: Linear-Quadratic Problems

$$V(T, x) = x' Q_T x$$

$$0 = \min_{u \in \mathbb{R}^{m \times m}} \left[x'Qx + u'Ru + x'\dot{K}(t)x + 2x'K(t)Ax + 2x'K(t)Bu \right]$$

Solve the minimization in u by setting the derivative to zero.

$$\nabla_u \left[x'Qx + u'Ru + x'\dot{K}(t)x + 2x'K(t)Ax + 2x'K(t)Bu \right] = 0$$

$$\implies 2B'K(t)x + 2Ru = 0$$

$$\implies \boxed{u = -R^{-1}B'K(t)x}$$

Example: Linear-Quadratic Problems

Substitute $u = -R^{-1}B'K(t)x$ into

$$0 = \min_{u \in \mathbb{R}^{m \times m}} \left[x'Qx + u'Ru + x'\dot{K}(t)x + 2x'K(t)Ax + 2x'K(t)Bu \right]$$

Example: Linear-Quadratic Problems

Substitute $u = -R^{-1}B'K(t)x$ into

$$0 = \min_{u \in \mathbb{R}^{m \times m}} \left[x'Qx + u'Ru + x'\dot{K}(t)x + 2x'K(t)Ax + 2x'K(t)Bu \right]$$

We get

$$0 = x' \left(\dot{K}(t) + K(t)A + A'K(t) - K(t)BR^{-1}B'K(t) + Q \right) x, \quad \forall (t, x)$$

Example: Linear-Quadratic Problems

Substitute $u = -R^{-1}B'K(t)x$ into

$$0 = \min_{u \in \mathbb{R}^{m \times m}} \left[x'Qx + u'Ru + x'\dot{K}(t)x + 2x'K(t)Ax + 2x'K(t)Bu \right]$$

We get

$$0 = x' \left(\dot{K}(t) + K(t)A + A'K(t) - K(t)BR^{-1}B'K(t) + Q \right) x, \quad \forall (t, x)$$

$= 0$

So, for $V(t,x) = x'K(t)x$ to solve the HJB equation, $K(t)$ must satisfy the Continuous-Time Riccati Equation:

$$\boxed{\dot{K}(t) = -K(t)A - A'K(t) + K(t)BR^{-1}B'K(t) - Q}$$

With terminal condition $\boxed{K(T) = Q_T}$

Example: Linear-Quadratic Problems

To get the optimal controller, start at the terminal condition $K(T) = Q_T$

Solve the Riccati equation backwards in time to get $K(t)$.

Plug $K(t)$ into $u^* = -R^{-1}B'K(t)x$ to get the optimal controller.

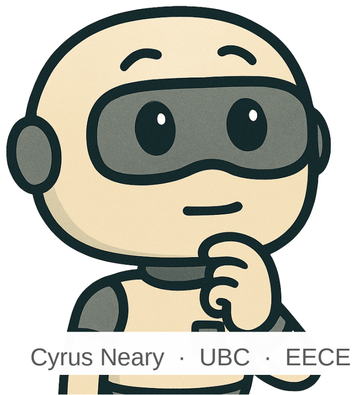
What about if we had solved the time-discretized version of the linear quadratic problem?

- we would have gotten discrete Riccati equations

$$K_T = Q_T$$

$$K_{T-1} = Q + A' K_t A - A' K_t B (B' K_t B + R)^{-1} B' K_t A$$

↳ start at the end and recursively solve this system of equations.



A numerical example

Double integrator system: $x = \begin{bmatrix} p \\ v \end{bmatrix}$ Position / velocity

$\dot{p} = v$ Time derivative of position IS velocity

$\dot{v} = u$ Acceleration is given by control input u

In matrix form:

$$\underbrace{\begin{bmatrix} \dot{p} \\ \dot{v} \end{bmatrix}}_{\dot{x}} = \underbrace{\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} p \\ v \end{bmatrix}}_{x} + \underbrace{\begin{bmatrix} 0 \\ 1 \end{bmatrix}}_{B} u$$

Set initial condition $x = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$, and cost components $Q = Q_T = \begin{bmatrix} 10 & 0 \\ 0 & 1 \end{bmatrix}, R = [1.0]$

goal: $x_T = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

cost u^2 for applying input u

$u^T R u$

$x^T Q x$

weigh position penalties heavily (10)

velocity penalties with factor (1)

A numerical example

```python
def double_integrator_matrices():
    A = np.array([[0.0, 1.0],
                  [0.0, 0.0]])
    B = np.array([[0.0],
                  [1.0]])
    return A, B


def riccati_rhs(A, B, Q, R, P):
    """
    Compute RHS of the *backward* Riccati ODE:
        -Pdot = A^T P + P A - P B R^{-1} B^T P + Q
    Return Pdot (for forward time), so the ODE is:
        dP/dt = -(A^T P + P A - P B R^{-1} B^T P + Q)
    """
    Rinv = np.linalg.inv(R)
    PB = P @ B
    BRB = PB @ (Rinv @ (B.T @ P))
    return -(A.T @ P + P @ A - BRB + Q)
```

```python
def riccati_finite_horizon(A, B, Q, R, S, T, num_points=1000):
    """
    Integrate the matrix RDE backward from t=T to t=0.
    Returns:
      t_grid (ascending),
      P_seq: (num_points, n, n),
      K_seq: (num_points, m, n)
    """
    n = A.shape[0]
    # Flatten P to integrate as a vector
    def f(t, Pflat):
        P = Pflat.reshape(n, n)
        # enforce symmetry numerically
        P = 0.5 * (P + P.T)
        dP = riccati_rhs(A, B, Q, R, P)
        # keep symmetry in derivative as well
        dP = 0.5 * (dP + dP.T)
        return dP.ravel()

    # integrate from T down to 0 (solve_ivp supports decreasing t_span)
    t_span = (T, 0.0)
    t_eval_desc = np.linspace(T, 0.0, num_points)
    sol = solve_ivp(
        f, t_span, S.ravel(), t_eval=t_eval_desc, method="RK45", rtol=1e-8, atol=1e-10
    )
    if not sol.success:
        raise RuntimeError(f"Riccati integration failed: {sol.message}")

    # reverse to ascending time for convenience
    t_grid = sol.t[::-1]
    P_flat_seq = sol.y[:, ::-1].T  # shape (num_points, n*n)
    P_seq = P_flat_seq.reshape(-1, n, n)
    # clean symmetry
    P_seq = 0.5 * (P_seq + np.transpose(P_seq, (0, 2, 1)))

    # K(t) = R^{-1} B^T P(t)
    Rinv = np.linalg.inv(R)
    K_seq = np.array([Rinv @ (B.T @ P) for P in P_seq])  # (N, m, n)

    return t_grid, P_seq, K_seq
```

A numerical example

```python
def simulate_closed_loop(A, B, K_t, t_grid, x0):
    """
    Simulate xdot = (A - B K(t)) x, u = -K(t) x using 4th-order Runge-Kutta.
    K_t: array of shape (N, m, n) corresponding to t_grid (ascending)
    """
    n = A.shape[0]
    m = B.shape[1]
    # interpolate each entry of K over time
    K_interp = interp1d(t_grid, K_t, axis=0, kind="linear", fill_value="extrapolate")

    def f(t, x):
        K = K_interp(t)  # (m, n)
        u = -K @ x
        return (A @ x + B @ u).ravel()

    x_traj = [x0.copy()]
    u_traj = []
    for k in range(len(t_grid) - 1):
        t, h = t_grid[k], t_grid[k + 1] - t_grid[k]
        x = x_traj[-1]

        # RK4
        k1 = f(t, x)
        k2 = f(t + 0.5 * h, x + 0.5 * h * k1)
        k3 = f(t + 0.5 * h, x + 0.5 * h * k2)
        k4 = f(t + h, x + h * k3)
        x_next = x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)

        # store
        K_now = K_interp(t)
        u_now = -(K_now @ x)
        u_traj.append(u_now)
        x_traj.append(x_next)

    x_traj = np.vstack(x_traj)           # (N, n)
    u_traj = np.vstack(u_traj + [u_traj[-1]])  # pad to length N
    return x_traj, u_traj
```

$$h(x_T) = x_T^T Q_T x_T$$

$$V(t,x) = x^T K(t) x$$

$$V(T,x) = h(x_T)$$

$$x_T^T K(T) x_T = x_T^T Q_T x_T$$

$$K(T) = Q_T$$

A numerical example

```python
# Problem data
A, B = double_integrator_matrices()
# Weights (tune as desired)
Q = np.diag([10.0, 1.0])   # penalize position more than velocity
R = np.array([[0.1]])      # control effort penalty
S = Q.copy()               # terminal cost
T = 5.0                    # horizon (seconds)

# Solve finite-horizon RDE
t, P_seq, K_seq = riccati_finite_horizon(A, B, Q, R, S, T, num_points=1501)

# Example: simulate closed-loop from some initial state
x0 = np.array([2.0, 0.0])  # start at position 2m, zero velocity
x_traj, u_traj = simulate_closed_loop(A, B, K_seq, t, x0)
```

